



Representation, Search and Verification

Compiler AX의 실험 설계와 조직 적용

Jihwan Ryu

류지환 · 지원자 작성 자료

목차

코딩 에이전트의 초안을 실제 변경으로 채택하려면, 통과 기록만으로 설명되지 않는 판단을 확인해야 합니다. 개인 개발 환경과 공개 Rust SDK의 CPU 실험을 출발점으로, 담당 엔지니어의 판단을 작은 절차와 검사에 담고 다음 업무에서 검증하는 Compiler AX 적용안을 제시합니다.

I	직접 만든 환경과 확인한 결과	
	1. Agent-Assisted Development	03
	개인 개발 환경은 어떤 문제를 해결하기 위해 만들어졌는가?	
	2. CPU Validation: Test Strengthening and Candidate Selection	04
	실행 결과와 코드 검토는 각각 무엇을 확인했는가?	
II	공개 스택에 맞춘 조직 적용	
	3. RNGD Architecture and the Compiler Stack	06
	TCP-컴파일러·Runtime의 책임은 어디에서 나뉘는가?	
	4. Compiler AX: Expert Review and Bounded Delegation	07
	무엇을 문서로 남기고, 무엇을 코드 검사로 강제할 것인가?	
III	채택과 조건부 확장	
	5. Verification, CI Ratchets, and Change Review	09
	검사·trajectory·PR을 어떤 판단에 연결할 것인가?	
	6. Adoption Scope and Technical Extensions	11
	Compiler 업무와 탐색 정책은 어떤 근거가 있을 때 확장하는가?	
	References and Evidence Scope	13
	직접 실행·공개 자료·미실행 제안의 원문과 경계	

2026년 7월 제출 원본을 보존한 후속 개정본입니다. 1-2절은 개인 개발과 9월 16일 완료한 CPU 사례, 3절은 공개 스택, 4-6절은 조직 적용과 확장 제안입니다. 5.2절은 공개 Lab의 CI 구현도 구분해 설명합니다. 전문가 관찰·팀 도입·NPU 실행은 미실행입니다.

1. Agent-Assisted Development

추론 모델과 코딩 에이전트가 빠르게 발전할 때, 기존 백엔드·인프라 지식으로 어디까지 제품을 직접 만들고 제어할 수 있는지 확인하고 싶었습니다. Cursor와 Claude 계열 도구로 시작해 Codex 중심으로 작업하면서, 구현을 실행해 둔 동안 논문과 코드를 읽고 결과를 다시 검사하는 방식이 자리 잡았습니다. 혼자 다룰 수 있는 범위가 넓어진 만큼 **무엇을 바꿨고 어떤 근거로 받아들였는지**를 남기는 일이 중요해졌습니다.

한 파일을 고친 뒤 같은 오류가 여러 경로에서 반복되면 파일별 수정 요청을 늘리기보다 원인을 다시 나눴습니다. 공통 의존성 설정의 문제인지, 도구의 성공 응답을 작업 완료로 잘못 읽었는지, 검사한 코드와 지금 코드가 같은 버전인지부터 확인했습니다. 관점을 바꾼 결과는 재현 입력·허용 변경·검사 명령으로 구체화했습니다. 문제를 더 추상적으로 말하는 데서 끝내지 않고, **어디에 개입하면 가설을 구별할 수 있는지**까지 내려오는 방식입니다.

1.1 Development Workflow and Evidence

작업은 요구와 관련 코드를 읽고 이미 구현된 부분을 제외하는 데서 시작합니다. Codex에는 남은 문제와 변경 범위를 주고, 독립적인 조사·구현은 나누되 같은 파일의 동시 수정은 피합니다. 초안 뒤에는 diff를 처음 요구와 대조하고 단위 검사·E2E·실제 화면을 필요한 범위에서 확인합니다. 검사 결과가 좋더라도 사용 예시나 문서가 다른 revision을 가리키면 완료로 두지 않습니다. GEODE의 개발 규약과 작은 회귀 검사는 이 과정을 다음 작업에서도 사용할 수 있게 만든 장치입니다.[1]

이 방식으로 개발한 GEODE와 Eco²에서는 도구·작업·이벤트의 성공 상태가 실제 완료와 어긋나는 문제를 만났습니다. 다음 사례는 그 차이를 실행 조건과 검사로 좁힌 과정입니다.

1.2 Runtime and Recovery: GEODE and Eco²

장기 도구 작업용 런타임 **GEODE**에서는 모델이 제안한 행동을 실행할지, 현재 작업을 계속할지 따로 판단하도록 했습니다. 필요한 skill과 작업 정보로 문맥을 조립하고, 도구 호출마다 허용 범위·예산·승인 조건을 확인합니다. 실행 결과를 다음 계획에 반영해도 같은 도구·입력·결과의 반복을 성공 응답으로만 읽으면 작업이 계속될 수 있었습니다. 이 고장에는 반복 조건을 종료 판단에 추가하고 해당 실행 경로의 mock 회귀 검사로 확인했습니다.[1]

분리배출 안내 서비스 **Eco²**에서는 AI 처리의 수명이 HTTP 연결보다 길었습니다. 작업 식별자, 비동기 실행, 순서가 있는 이벤트 이력, 실시간 전달을 분리해 연결 종료와 실제 작업 완료를 구별했습니다. 복구 경로에서는 전달에 실패한 이벤트까지 ACK해 재회수 대상에서 빠지는 문제를 찾았습니다. Consumer와 Reclaimer가 성공한 처리만 ACK하도록 수정하고 같은 성공·실패·예외 조건으로 전후 동작을 대조했습니다. 확인 범위는 Consumer와 Reclaimer의 이벤트 전달 복구입니다.[2]

두 사례에서 공통으로 바뀐 것은 성공을 읽는 방법이었습니다. 도구가 응답했는지와 목표가 진전됐는지, 전달을 시도했는지와 완료했는지를 나눠야 다음 행동을 정할 수 있었습니다. 개별 고장을 재현하는 검사에 더해, 반복 탐색이나 오래된 문맥 유지가 어떤 상황에서 나타나는지 살필 평가 환경도 구성했습니다.

1.3 Simulation and Workflow Evaluation

GEODE의 **SIL**은 이런 행동을 시험할 상황을 만들고 지침 변경의 결과를 확인하도록 구성했습니다. Seed Scenario Generation에서 생성한 상황은 비평가 pilot으로 좁혔고, Simulation에서는 Petri의 auditor·target·judge 상호작용으로 행동을 관찰했습니다. 지침 변경은 autoresearch 방식으로 한 항목씩 비교하고 재평가 결과에 따라 유지하거나 복원하도록 했습니다. 기각 뒤에도 설정이 남던 문제를 고칠 때에는 판정 기록과 함께 실제 설정의 복원 상태를 검사했습니다.[3]

지침을 고르는 데 쓴 관측에 맞춰 최종 확인 기준까지 바꾸면 새 후보의 개선을 구분하기 어렵습니다. **Crucible**은 개발 중의 관측으로 다음 후보를 고르되 최종 확인의 입력과 판정 기준을 고정하도록 구성했습니다. 일부 시나리오는 모의 도구 응답을 사용하고 재평가 실행은 선택 사항이므로, 각 후보의 채택 근거는 실제로 실행한 검사와 결과에 둡니다.

	관측한 문제	다시 물은 질문	수정한 조건 · 확인 근거
GEODE 장기 도구 작업	성공 응답을 받아도 같은 조회를 반복	성공했는가보다 관측이 달라졌는가?	동일 도구·입력·결과의 반복을 탐지해 종료 실제 실행 루프 + mock 회귀 모든 의미적 무진전의 판정은 아님
Eco² 이벤트 전달	전달 실패 뒤 ACK 재처리 경로에서 제외	시도를 마쳤는가보다 전달에 성공했는가?	성공한 처리만 ACK 실패는 pending 유지 Consumer 전후 코드 · 격리 비교 Redis·서비스 통합 재측정은 아님
SIL 설정 후보 평가	후보를 기각했지만 바뀐 설정은 잔류	기각을 기록했는가보다 이전 상태로 돌아갔는가?	이전 값을 복원하고 새로 넣은 항목은 제거 실제 복원 helper · 격리 테스트 개선 점수·무인 배포의 증거는 아님

2026-09-12 감사의 확인 범위 · 사람과 코딩 에이전트가 고친 사례

붉은 선: 수정한 완료 조건

그림 1. 성공 응답·전달 시도·기각 기록이 실제 완료를 뜻하지 않았던 세 사례입니다. 각 차이를 종료·ACK·설정 복원 조건과 해당 검사로 바꿨습니다. 서로 다른 프로젝트의 독립된 사례입니다.

비교 환경도 검사 대상이었습니다. GEODE와 native Codex의 Terminal-Bench 2.1 비교에서는 매번 작성하던 샌드박스·스크립트 대신 Harbor로 과제·검증기·산출물 수집을 맞췄습니다. 89개 과제에 양쪽 각 5회, 총 890셀을 계획했지만, 9월 12일 분석에는 검증기 호환성 처리 뒤에도 인프라 무효가 남았습니다. 전체 비교는 측정 불성립으로 정리했습니다. **외부 하네스를 채택해도 환경의 실패를 에이전트 실패와 구분하는 책임은 남습니다.**[4]

이 경험을 Compiler AX Lab에 적용하면서는 테스트의 실행 여부와 함께 **그 테스트가 의도한 오류를 구별할 수 있는지를** 물었습니다. 다음 CPU 사례는 이 질문에 맞춰 보강한 검사와, 같은 통과 결과 뒤에 남은 코드 선택의 근거를 보여 줍니다.

2. CPU Validation: Test Strengthening and Candidate Selection

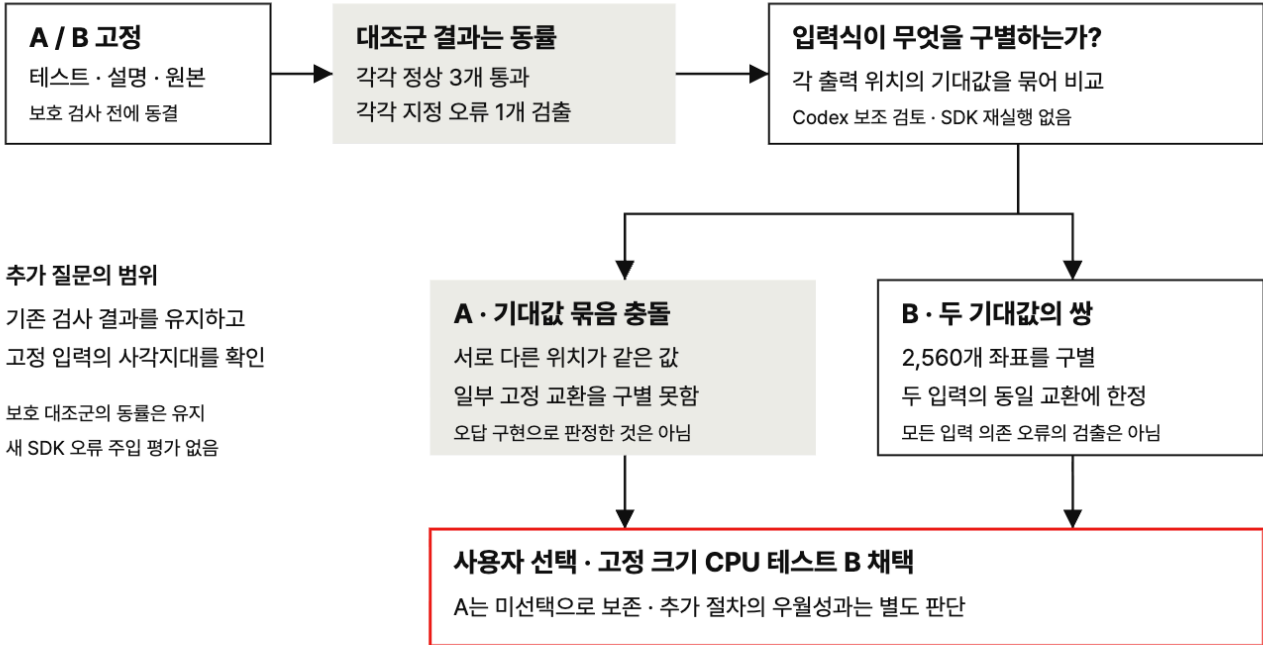
공개 `furiosa-opt` 의 double-buffering 실행만으로 그룹별 출력이 올바른지는 알 수 없었습니다. 첫 과제는 제품 커널 최적화가 아니라 **테스트의 판정 능력 보강**으로 좁혔습니다. 독립 scalar reference와 좌표별 비교를 추가하고, 정상 구현은 통과하면서 의도한 오류는 수치 assertion에서 잡는지 확인했습니다. 후보가 제품 코드나 보호된 평가 기준을 바꾸는 것은 허용하지 않았습니다.[5]

2.1 A/B Controls and Code Review

A에는 상세 공통 요구를, B에는 같은 요구와 조사·진단·수정·검증 절차를 제공했습니다. 공개 코드와 도구는 같았고 초안 뒤 수정은 한 번으로 제한했습니다. 두 코드와 설명을 모두 고정한 뒤 보호된 정상·오류 구현을 검사했으며 그 결과를 다시 후보 수정에 주지 않았습니다. 한 과제의 한 쌍이므로 절차 효과의 일반화보다 실행 가능성과 검토할 차이를 확인하는 파일럿입니다.

한 과제에서 고정 후보 2개와 구현 4개를 조합한 **8개 판정 단위 중, 각 후보는 정상 구현 3개를 통과시키고 자격이 확인된 오류 1개를 지정 수치 assertion에서 검출했습니다.** 이 대조군에서 관측된 오탐·미탐은 모두 0입니다. 미관측 오류에 대한 검출 능력은 후속 입력과 소스 검토로 확인할 질문으로 남았습니다.[5]

실행 완료 → 별도 코드 검토 → 사용자 선택



사용자 Rust 전문성·Furiosa 승인·NPU 검증은 뜻하지 않습니다.

붉은 테두리: 사용자 선택

그림 2. 같은 검사를 통과한 사실과 특정 코드를 채택한 이유는 다릅니다. 후속 소스 검사는 선택 근거이며 A/B의 추가 실행 표본이 아닙니다.

후속 검토에서는 입력이 서로 다른 출력 위치를 구별하는지 물었습니다. A의 여섯 입력에서 각 좌표의 기대값을 순서대로 묶으면, 2,560개 좌표 중 서로 다른 묶음은 2,332개였습니다. 서로 다른 두 좌표가 같은 묶음을 갖는 경우, 그 두 위치를 항상 교환하는 오류는 해당 입력만으로 구별할 수 없습니다. 정상값을 더 많이 계산했다는 사실과 위치를 더 잘 식별한다는 성질은 달랐습니다.

B는 출력의 일렬 번호를 251로 나눈 나머지와 묶음 각각 1을 더한 값을 두 입력의 기대값으로 사용했습니다. 두 값을 함께 읽으면 2,560개 좌표가 모두 구별됩니다. **같은 위치 교환이 두 입력에 적용되는 오류**를 드러낼 수 있고, 입력·기대값이 bf16으로 정확하게 표현되는 정수 범위를 벗어나면 중단하는 검사도 명시했습니다. 반면 A는 기저 입력으로 reduction 축을 더 넓게 탐침하고 실패 메시지가 자세합니다. B의 기저 탐침은 좁으며 dense 입력으로 전체 축을 확인하므로, 모든 결함에 우월한 후보로 순위를 매기지는 않았습니다.

9월 16일 Codex의 보조 검토를 바탕으로 **B를 고정 크기 CPU 테스트 보강안으로 채택**하고 A를 비교 후보로 보존했습니다. 확인 범위는 CPU의 수치 결과와 입력 식별성입니다. 실제 HBM 할당 재사용과 DMA 중첩은 장치 실행에서 확인할 항목입니다.[5]

이번 파일럿에서 테스트를 보강하고 채택할 코드를 선택했습니다. 두 후보의 대조 결과는 같아 추가 절차의 효과는 확인하지 못했습니다. 생성 순서가 B 다음 A로 고정됐고 계정·캐시 조건도 변했으며 사람의 총 작업시간은 측정하지 않았습니다. 다음 절차 비교에서는 이 조건을 맞추고 검토·재작업 시간을 함께 기록하겠습니다.

2.2 Validation Scope and Limitations

수치 출력의 정확성을 확인한 A/B와 별도로, 입력 문법이 구문 구조와 진단 위치로 올바르게 이어지는지도 검사했습니다. Mapping parser의 두 진입점에서 문법→AST→오류 문구·byte 위치를 대조하도록 검사를 6개에서 12개로 늘렸습니다.

이 parser 검사가 분리된 빌드에서도 작동하는지 확인하려고 원본·보강본·공개 오류 사본의 소스를 다시 꺼내 각각 다른 빌드 디렉터리에서 재현했습니다. 정상 검사는 통과하고 오류 사본은 지정 assertion에서 실패했습니다. 의존성과 toolchain은 재사용했으므로 완전히 새 환경의 재현과는 구분합니다. 앞선 환경 권한 실패는 테스트 전에 멈춘 무효 실행으로 보존했습니다.

앞서 공개한 double-buffering 테스트와 parser 변경을 함께 적용한 별도 CPU 통합 검사에서는 개별 변경을 같은 환경에 놓았을 때 기존 검사까지 통과하는지 확인했습니다. 이 실행의 대상은 B가 아니며 A/B의 추가 표본으로 합산하지 않았습니다. 상세 검사 수와 환경, 별도로 수행한 native x86 smoke-합성 상태 복원은 후주 5에 둡니다.[5]

남은 질문은 담당자의 실제 판단을 절차에 담았을 때, 다른 과제에서 필요한 검사를 빠뜨리지 않고 검토·재작업 부담을 줄일 수 있는 가입니다. 다음 절부터는 공개 스택의 책임을 나눈 뒤 이를 조직에서 확인할 적용안을 제시합니다.

3. RNGD Architecture and the Compiler Stack

RNGD에서는 같은 계산도 데이터를 어디에 남기고 언제 옮기는지에 따라 비용이 달라집니다. TCP(Tensor Contraction Processor)는 다양한 텐서 모양의 병렬성과 재사용을 활용하도록 설계했습니다. Compiler는 이 유연성을 텐서 배치와 실행 순서로 구체화합니다. 자동 생성량보다 먼저 살필 것은 현재 표현과 관측으로 어떤 선택을 구별할 수 있는지입니다.[6]

물리 데이터 흐름. HBM과 PE의 SRAM 사이에서는 Tensor DMA가 데이터를 옮깁니다. SRAM의 값은 fetch 경로를 거쳐 수축·벡터·전치 연산기에 전달되고, 결과는 commit 경로로 SRAM에 기록됩니다. 값을 재사용하면 HBM 왕복을 줄일 수 있지만, 공간을 오래 점유하면 다른 작업의 배치가 제한됩니다. 한 연산이 빨라져도 인접 연산의 layout 변환과 buffer 수명이 달라지면 전체 실행은 느려질 수 있습니다. 따라서 연산·텐서의 의존성과 실제 메모리·실행 순서를 함께 살펴야 합니다.[6]

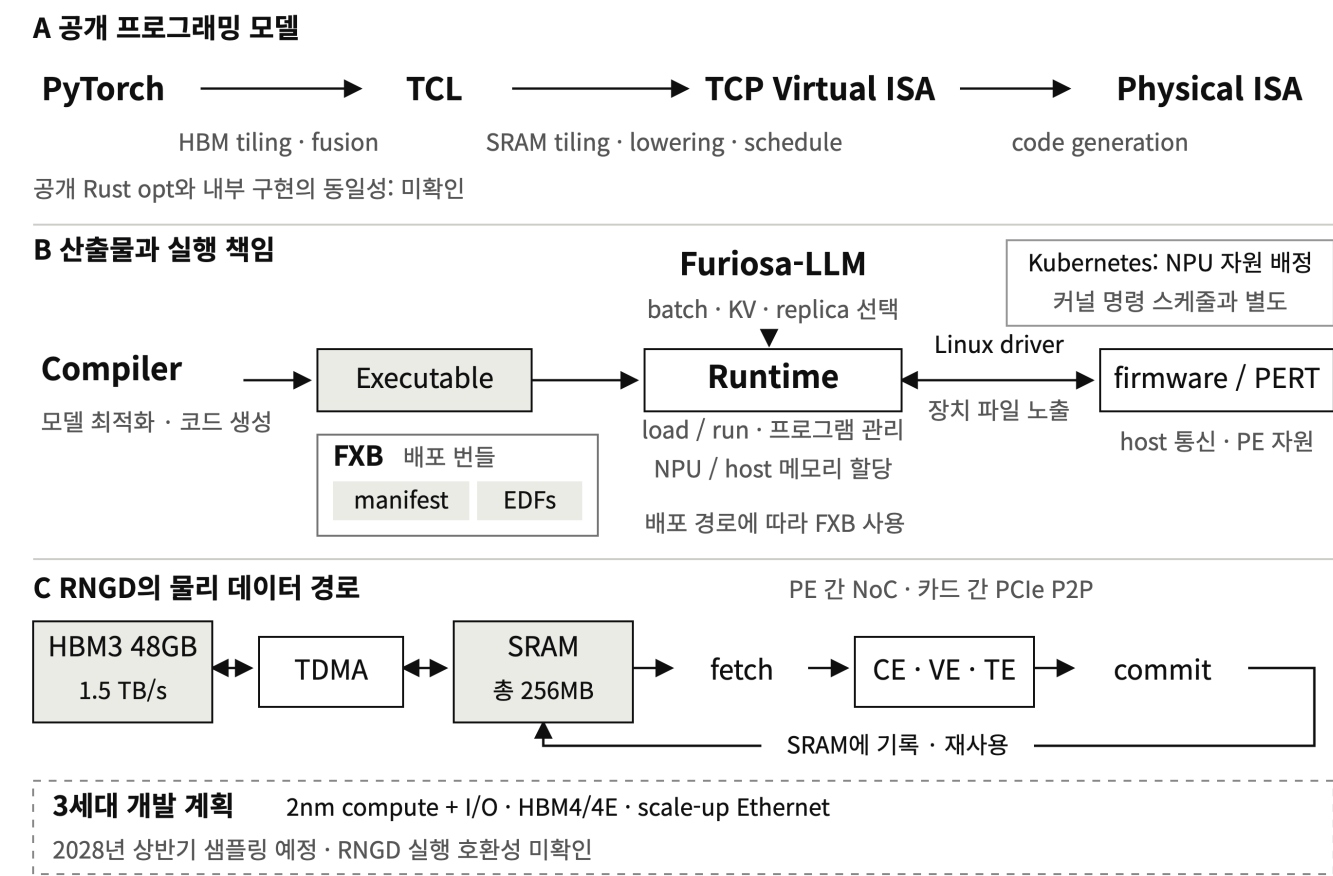


그림 3. 계산 표현·컴파일 산출물·장치 실행을 구분해야 같은 지연 현상에서도 수정할 계층을 찾을 수 있습니다. 공개 자료 재구성이며 3세대는 개발 계획입니다.[6-8]

3.1 Programming Model and Semantic Constraints

공식 발표의 **PyTorch → TCL → TCP Virtual ISA → TCP Physical ISA**는 계산 의도를 점차 실행 가능한 형태로 바꾸는 계층입니다. TCL은 tensor contraction의 의미를 남기고 compiler가 tiling·fusion·mapping·scheduling을 선택하도록 합니다. 공개 Rust `furiosa-opt`는 관찰 가능한 커널 작성·검사 경로지만, 공개 driver는 설치 안내 stub이므로 production compiler의 내부 pass 구현까지 이 저장소에서 수정할 수 있다고 전제하지 않습니다.[7]

변환의 기준은 명령 형태의 동일성이 아니라 요구된 값·효과·순서입니다. Buffer를 재사용하려면 이전 값을 읽는 소비가 끝난 뒤 덮어써야 합니다. 직렬 host 실행에서 값이 맞아도 장치에서 동시에 실행할 때의 수명과 순서가 맞는지 별도 확인이 필요합니다. 변경이 테스트 보장인지, 의미를 보존하는 최적화인지, 잘못된 동작을 바꾸는 bug fix인지 먼저 정해야 기준도 맞게 세울 수 있습니다.[9]

Compiler가 만든 executable은 Runtime이 읽어 프로그램과 메모리를 관리하며 실행합니다. Driver·firmware·PE Runtime은 장치 실행을 받치고, Furiosa-LLM은 모델과 요청을 제공합니다. FXB의 fingerprint·compiler revision 일치는 산출물 호환성 확인입니다. 요청의 품질·지연·처리량은 실제 workload에서 다시 봐야 합니다. x86-64 요구도 배포 host toolchain의 조건이지 NPU의 텐서 명령 집합을 뜻하지 않습니다.[8]

3.2 Model Integration and Serving Operations

서빙 개선의 원인을 파악하려면 컴파일 산출물을 바꾸는 개입과 같은 산출물의 실행 조건을 바꾸는 개입을 나눠야 합니다. Bucket 구성을 바꾼다면 모델·compiler 버전·나머지 설정·측정 workload를 고정하고 후보별 FXB를 다시 만들어 비교하겠습니다. KV runtime 설정만 바꿀 때는 같은 FXB와 입력·출력 길이·동시성에서 확인하겠습니다. 공개 모델 적용 자료는 이런 변경 지점을 살필 근거이며, 공급사가 발표한 집계 성능에서 각 변경의 기여율을 구하려면 별도 비교가 필요합니다.[10]

지연이 모델 실행 전의 장치 배정에서 생겼다면, compiler patch를 만들어도 그 대기는 줄지 않을 수 있습니다. 공개 NPUaaS와 서빙 운영 자료를 참고해 장치 자원 배정·세션 수명·요청 실행을 나누어 살피겠습니다. 실제 업무에서는 담당자와 지연·실패가 발생한 층을 확인한 뒤 수정 범위를 정하겠습니다.[11]

공개 스택은 변경의 기술적 영향과 필요한 검사를 정하는 기준입니다. 첫 AX 적용 업무를 고르려면 이와 함께 담당자가 준비·검토·재작업에 시간을 쓰는 지점을 확인해야 합니다. Compiler AX 공고도 개발·리뷰·디버깅·CI/testing·문서와 도구 정착의 개선을 실제 팀 업무에서 다루도록 제시합니다. 다음 절에서는 이 업무 관측을 작은 절차와 제한된 위임으로 옮기는 방법을 제안합니다.[12]

4. Compiler AX: Expert Review and Bounded Delegation

개인 프로젝트에서 정한 절차를 팀 업무에 적용하려면 **해당 업무에서 보존할 조건과 판단을 멈출 근거**를 담당자에게 확인해야 합니다. 첫 적용에서는 실제 엔지니어의 한 작업을 함께 따라가겠습니다. 기존 patch 검토, 새 parser 경계 사례, 반복되는 테스트 준비, 문서 예제 불일치 가운데 실제 지연과 재작업이 관찰되는 일을 고릅니다. 현재 완료한 parser 사례를 새 독립 과제로 다시 세지는 않습니다.

4.1 Expert Observation and Procedure Design

담당 엔지니어가 열어 본 코드·진단, 선택한 검사, 버린 가설, 판단을 보류한 조건을 기록합니다. 가능하면 에이전트의 제안을 보기 전 판단을 먼저 남겨 도구가 검토자의 생각을 유도하는 영향을 구분합니다. Compiler 담당자는 의미·호환성·reference·허용 오차와 채택을 소유하고, AX 엔지니어는 반복 절차·도구 연결·사용 비용을 다룹니다. 한 사례의 취향을 모든 업무의 필수 규칙으로 만들지 않습니다.

기록 전체를 지침에 넣으면 다음 작업에 필요한 조건이 묻힙니다. MD에는 **적용할 문제, 필요한 입력과 revision, 수행 순서, 기대 관측, 적용하지 않을 조건, 중단 후 문의할 담당자만** 남깁니다. 결과가 기계적으로 판별되는 조건은 기존 script나 test에 넣고, 명세 해석이 필요한 부분은 검토 항목으로 둡니다. 담당 엔지니어가 MD와 검사의 적용 범위를 검토·승인한 뒤, 그 범위의 재현·증거 수집

·국소 수정만 에이전트에 맡깁니다. MD는 실행 선택을 안내하고 검사는 실제 결과를 판정하도록 역할을 나눕니다.

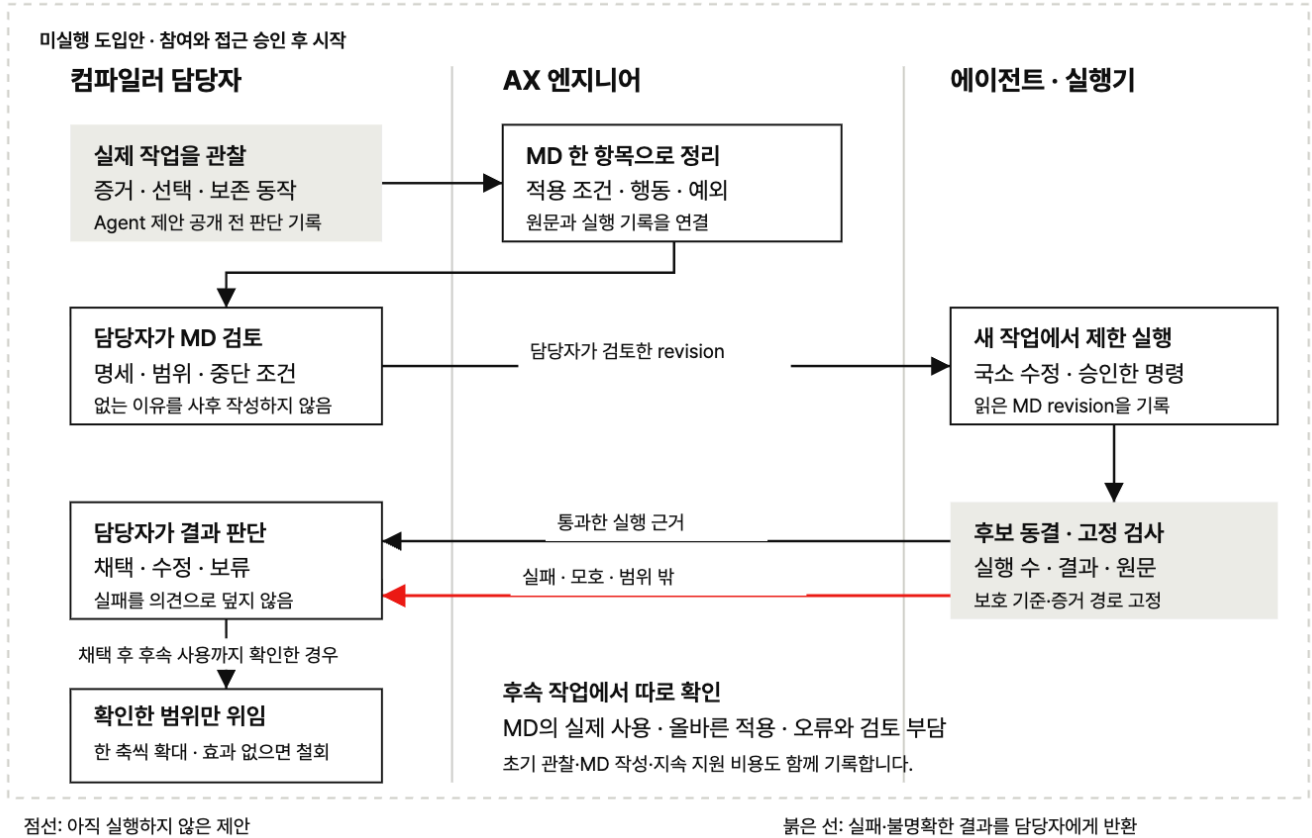


그림 4. 전문가의 판단을 기록한 뒤, 다음 작업에서 올바르게 사용되는지 확인하고 위임 범위를 정합니다. 현재 CPU 사례와 구분되는 미실행 조직 적용안입니다.

예를 들어 진단 테스트의 오류 위치를 보강한다면, 문법 입력만 더 생성하게 하지 않습니다. 담당자가 위치 단위와 기대 AST를 정한 뒤 에이전트가 누락된 경계 입력을 제한하고 두 parser 진입점의 결과를 대조하게 합니다. 기대 메시지를 구현 출력에서 그대로 복사하면 같은 오류를 reference에 심을 수 있으므로, 원래 문법 계약과 독립 예제로 확인합니다. 명세가 불명확한 입력은 성공·실패 중 하나로 억지 분류하지 않고 담당자에게 돌립니다.

4.2 Task Planning and Patch Review

새 문제에서는 입력 범위 누락·공통 설정·결과 해석 가운데 어떤 가설을 구별할지 정하고, 필요한 관측을 얻을 실행부터 고릅니다. 그 비교에 서로 다른 수정안이 필요하고 독립적으로 시험할 수 있을 때 별도 작업공간에서 후보를 병렬로 만듭니다. 독립 실행 여부는 수정할 파일뿐 아니라 공유 설정·장치·검사 자원의 충돌 가능성까지 보고 정하겠습니다. 기존 patch가 있다면 재현·검토·보완부터 시작합니다. 후보를 준비하고 비교하는 비용까지 포함한 효과는 다음 과제에서 확인하겠습니다.

시나리오 생성자는 빠진 입력과 실패 조건을 제안합니다. **실제 build/test는 제안된 입력을 실행하고 담당자가 정한 reference와 결과를 대조해 판정합니다.**[13]

기본 경로는 baseline 재현→허용된 국소 변경→변경 대상 검사→회귀 검사→리뷰입니다. 실패하면 에이전트는 오류 위치·기대값과 실제값·현재 diff로 원인을 다시 좁힙니다. 환경 실패에는 코드 수정을 요청하지 않고 환경을 복구하며, 기준 자체가 잘못됐다면 후보 작업을 멈추고 평가 기준을 별도 변경으로 검토합니다. 같은 오류·가설·코드가 반복되거나 예산이 끝나면 미해결 조건을 남기고 종료합니다.

4.3 Procedure Reuse and Adoption Criteria

첫 절차가 생긴 것과 재사용에 성공한 것은 다릅니다. 새 과제에서는 기존 작업 방식과 변경할 절차의 revision을 먼저 고정하고, 실제로 읽힌 MD 조항과 적용 제외 조건을 기록합니다. 비교에는 같은 task 유형·공통 요구·모델·도구·검사 기준·예산을 사용하고 생성 순서와 환경 차이를 남깁니다. 시작 전에 업무에 맞는 이익 지표 하나와 필수 정확성·검사 노력 방지 조건을 정합니다. 유지·적용 범위 축소·폐기·판정 보류의 조건도 결과를 보기 전에 담당자와 합의합니다.

시간은 준비·생성·실행 대기·사람 검토·재작업으로 나눠 기록합니다. 병렬 생성 뒤 통합 대기과 중복 수정이 늘면 전체 완료는 늦어질 수 있습니다. 반대로 실제 결함을 더 찾아 검토 시간이 늘었다면 시간만으로 절차를 폐기하지 않습니다. 사전에 정한 이익과 안전 조건으로 판단하고, 표본이나 비교 조건이 부족하면 효과 판정을 보류합니다. 올바르게 사용된 사례와 개선 효과가 확인된 사례를 구분하겠습니다.

5. Verification, CI Ratchets, and Change Review

Compiler 개발에서 통과한 관측은 판단의 시작이기도 합니다. 현재 소스의 바이너리인지, 필요한 테스트가 실제 실행됐는지, 정상 구현을 잘못 거절하지는 않는지 확인해야 결과를 사용할 수 있습니다. 이번 CPU 사례도 값 검사 뒤에 입력 식별성을 다시 검토했습니다. 팀 적용에서도 검사기 자체의 판정 능력과 실행 환경의 유효성을 분리하겠습니다.

5.1 Failure Classification and Test Validity

검사의 실패는 언제 결함 검출로 해석할 수 있습니까? 먼저 빌드·실행이 성립했는지, 어떤 결과를 기대한 검사인지 나눠야 합니다. 아래 4분면의 대상은 실행이 성립한 CPU 수치 검사입니다. 양성은 지정 수치 assertion이 실패해 검사가 결함을 보고한 경우이며, 실제 정상·결함 여부와 나눠 분류합니다.[14]

적용 대상	현재 보유한 근거	판정 위치
CPU 수치 검사	고정 A/B 각 정상 3건·결함 1건	아래 4분면. 단위는 후보×구현
컴파일 거절이 기대 동작인 검사	별도 통합 실행의 <code>compile_fail doctest</code> 44개	기대한 거절이면 해당 doctest 통과. A/B에 합산하지 않음
후보 코드의 빌드 오류	수정 전 B 초안의 <code>E0599</code>	빌드 FAIL·후속 수치 검사 NOT_RUN. 최종 집계 제외
준비·실행 불성립	별도 parser 재현의 초기 준비 실패	무효 실행으로 보존. 수치 검출로 세지 않음
정적 schedule·NPU·서빙 성능	이번 파일럿에서 미측정	별도 실행·판정 필요. CPU 결과로 대체하지 않음

§2의 고정 A/B 결과를 로그·소스 대조와 Codex 보조 검토로 분류했습니다. 대조군의 자격은 사전에 정한 검사 계약으로 확인했습니다.[5]

검사가 내린 결과	정상 대조군	자격이 확인된 결함 대조군
통과	TN: A 3 · B 3 정상 구현 통과	FN: A 0 · B 0 결함 미검출 시 입력·reference·assertion 점검
지정 수치 assertion에서 실패	FP: A 0 · B 0 정상 구현 거절 시 기준·허용 범위 점검	TP: A 1 · B 1 의도한 결함 검출

8개 단위의 관측이며 일반적인 검출률 추정이 아닙니다. 예상 밖 빌드 실패, timeout, 0개 실행, 무관한 panic은 수치 검출로 세지 않습니다. 후보의 빌드 회귀와 환경 실패를 별도 진단하고, 실행되지 않은 후속 검사는 NOT_RUN으로 남깁니다.

수정한 결함을 다음 변경에서도 검출하려면 재현 입력과 기대 동작을 회귀 검사로 남겨야 합니다.

5.2 Regression Tests and CI Ratchets

조직 적용에서는 결함이 있는 판본에서 실패하고 수정본에서 통과하는 최소 검사를 만든 뒤, 담당자가 명세와 적용 범위를 확인하도록 하겠습니다. 승인한 검사는 관련 경로의 후속 PR에서 필수로 실행합니다. **CI ratchet은 검토한 회귀 검사를 후속 변경의 필수 조건으로 유지해 같은 결함을 다시 검출하는 절차입니다.** 기존 자체에 결함이 있으면 근거와 대체 검사를 별도 검토하며, 후보가 테스트 삭제나 허용 오차 완화로 통과하게 하지는 않습니다.

CI는 검사 실행과 결과 집계를 강제하며, 회귀검사의 의미와 삭제·완화 여부는 담당자의 변경 검토로 다룹니다. 검사기나 reference를 고치면 영향을 받은 이전 결과를 식별합니다. 과거 기록은 보존하되, 바뀐 공통 기준으로 다시 검사하기 전에는 이전 PASS를 새 비교·채택의 근거에서 제외합니다.

공개 Compiler AX Lab에는 이 절차를 받치는 고정 검사와 CI 게이트를 구현했습니다. Python 검사는 최소 16개 실행과 skip 없음을 요구하고, 변경 경로로 선택된 필수 job이 실패·취소·누락되거나 예상과 달리 건너뛰어지면 최종 lab-ci가 실패합니다. Rust CI는 독립 reference의 검사와 고정 소스에 대한 patch 적용 가능성까지 다루며, SDK 통합·NPU 실행은 별도입니다. 새 실패의 테스트 편입은 검토된 변경으로 수행하며 자동 생성·등록 기능은 두지 않았습니다.[5]

코드 변경에는 기존 Rustfmt·Clippy·단위·통합 검사와 해당 API·문서 확인을 연결합니다. 독립 reference도 입력 범위·정밀도·인덱스 계산을 검토해야 합니다. Static compile, CPU 값, schedule, 실제 장치 측정은 서로 다른 성질을 확인하므로 작업이 겨냥한 위험에 맞게 선택합니다.[7]

미실행 AX 제안

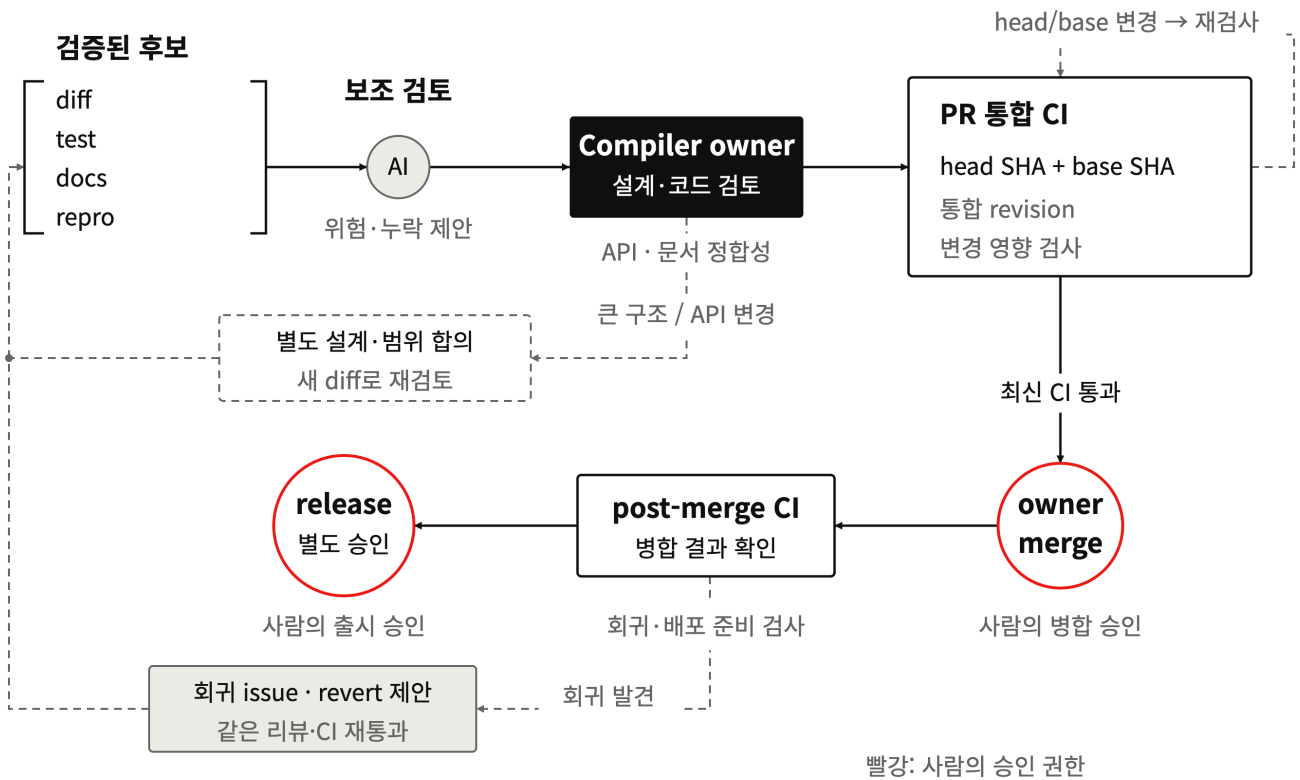


그림 5. 검토한 변경과 CI가 검사한 revision을 맞춘 뒤 병합합니다. 문서·예제·복구 경로까지 확인하되, 병합과 배포 승인은 구분합니다. 조직 적용 제한입니다.

5.3 Trajectory and Artifact Provenance

필요한 기록은 에이전트의 장황한 사고 설명이 아니라 **요청·기준 revision→관측과 가설→명령·도구 결과→diff와 산출물→검토 결정**입니다. Codex의 도구 호출·결과와 Git diff, 기존 검사 로그를 사용하고 작업·turn·call 식별자로 연결합니다. 원문은 보존하되 검토용 요약은 현재 revision·핵심 실패·미해결 조건·원문 위치를 우선 제공합니다. 중단된 명령은 남은 프로세스·파일·자원까지 확인한 뒤 재시도합니다.

해시는 파일의 동일성을 확인하지만 실행 조건과 결과의 의미까지 설명하지는 않습니다. 전체 로그를 큰 요약이나 객체 하나로 묶으면 필요한 구간을 찾는 비용도 커집니다. 검토자가 요약에서 원문으로 이동할 수 있도록 실행별 manifest에 생산 명령·revision·원문 위치·완전성·hash를 함께 남기겠습니다. 원문은 단계별로 나눠 조회하되 결합할 때 같은 작업의 기록인지 확인합니다. 기존 파일과 색인으로 충분한 동안 새 데이터베이스는 도입하지 않습니다.[15]

재현의 완료도 hash 일치로 끝나지 않습니다. 같은 명령과 조건에서 의미 있는 검사가 실제 실행됐는지, 결과가 필요한 산출물에 연결됐는지 확인합니다. 누락된 call/result, 잘린 출력, 불명확한 종료기 있으면 확인된 범위와 빠진 범위를 나눕니다. 원문 보존 범위는 접근 권한과 보존 기간으로 관리하며, 보호된 평가나 계정 정보는 공개 PR에 실지 않습니다.

5.4 PR Integration and Merge Approval

검사 결과가 있어도 API·문서·예시가 바뀐 동작을 따라가지 못하면 동료가 사용 조건을 다시 확인해야 합니다. PR에서는 앞 절의 실행 근거와 함께 코드·문서·예제의 불일치를 검토하겠습니다. 실제 환경의 검증부터 동료가 사용할 제품까지 다루는 Dioxus의 운영 회고를 이 완료 범위의 참고로 삼았습니다.[16]

PR에는 문제와 재현 조건, 선택한 수정과 버린 대안, 보존할 동작, 실행한 검사 수와 결과, 남은 판단을 담습니다. LLM은 설명과 누락을 점검할 수 있지만 실패한 코드 검사를 뒤집지 못합니다. 담당자는 의미·호환성·영향 범위를 검토하고, base나 head가 바뀌면 현재 조합을 다시 검사합니다. 저장소 병합은 합의된 Git 흐름을 따르며, target 실행 승인과 배포는 별도 결정으로 남깁니다.

한 작업의 완료는 재현한 문제에 대응하는 diff, 필요한 검사, 담당자의 검토, 동료가 실행할 예제·문서가 같은 revision에서 맞는 상태입니다. 실제 팀에서는 동료가 이를 다시 사용하는 비용까지 확인하겠습니다. 이 조건을 만족하는 변경을 적은 재작업으로 전달하겠습니다.

6. Adoption Scope and Technical Extensions

§5의 완료 조건을 갖춘 뒤에는 어떤 업무에 절차를 재사용할지, 다음 판단에 어떤 검증이 더 필요한지 정하겠습니다. **자동화 권한과 기술 범위는 나누어 조정합니다.** 반복 사용의 효과는 §4.3의 기준으로 확인하고, 기술 범위를 넓힐 때에는 그 질문에 필요한 실행 환경과 검사를 추가하겠습니다.

6.1 Host, Compiler, and Device Validation

저장소 업무부터 반복 사용을 확인합니다. 실제 담당자가 선택한 새 유지보수 과제에서 재현·국소 수정·리뷰·문서 중 한 범위를 맡습니다. 검토된 MD가 올바르게 사용되고 필요한 검사·문기가 유지됐는지 확인한 뒤, §4.3에서 미리 정한 기준에 따라 인접 작업으로 넓히거나 적용을 제한합니다. 사용 기록만 있고 효과가 불분명하면 확장을 보류합니다. 온보딩과 Q&A에서는 동료가 적용 경계와 중단 조건을 이해했는지 확인하겠습니다.[12]

컴파일과 schedule은 실행 순서나 자원 선택의 효과를 확인할 때 연결합니다. 호환 도구와 compiler binary를 확보하고, 담당자가 변경 가능한 범위를 정하면 같은 조건의 baseline과 candidate를 비교합니다. 공개 double-buffering 예제에서도 반복을 유지하거나 두 묶음씩 결합하거나 완전히 펼치면 스케줄러가 함께 배치할 연산의 범위가 달라집니다. 대기 구간이 의존성 때문인지 공유 자원 때문인지 구분한 뒤 한 선택을 바꾸겠습니다. 내부 lowering 수정에는 해당 소스·명세·검사가 접근을 갖추겠습니다.[17]

정적 계획이 좋아진 후보는 실제 장치에서 확인하겠습니다. 정적 비용으로 제외한 후보에서도 실제 개선을 놓칠 수 있으므로, 필요하면 승인된 예산 안에서 일부 대조 후보를 함께 확인합니다. 값의 정확성과 속도는 별도로 판단해, 빠른 실행이 수치 오류를 가리지 않게 하겠습니다.[7][17]

물리 RNGD와 모델 서빙은 접근 조건을 갖춘 별도 실행입니다. 장치·driver·firmware·runtime과 workload를 고정하고 값·의존성·반복 timing을 확인합니다. 모델 업무에서는 FXB 호환성 뒤에 출력 품질을, 서비스 업무에서는 같은 입력·출력 길이·동시성·cache 조건에서 TTFT·TPOT·실패·처리량을 확인합니다. 기존 FXB의 서빙 검증은 자체 커널 변경 없이 시작할 수도 있습니다. 새

칩으로 옮길 때에는 공개 개발 계획을 바탕으로 살필 조건을 정하고, 실제 명세·SDK·장치가 제공된 뒤 target별 검사를 다시 구성하겠습니다.[8][18]

기술 범위는 작업·자원에 따라 선택하고, 위임 권한은 별도로 승인합니다

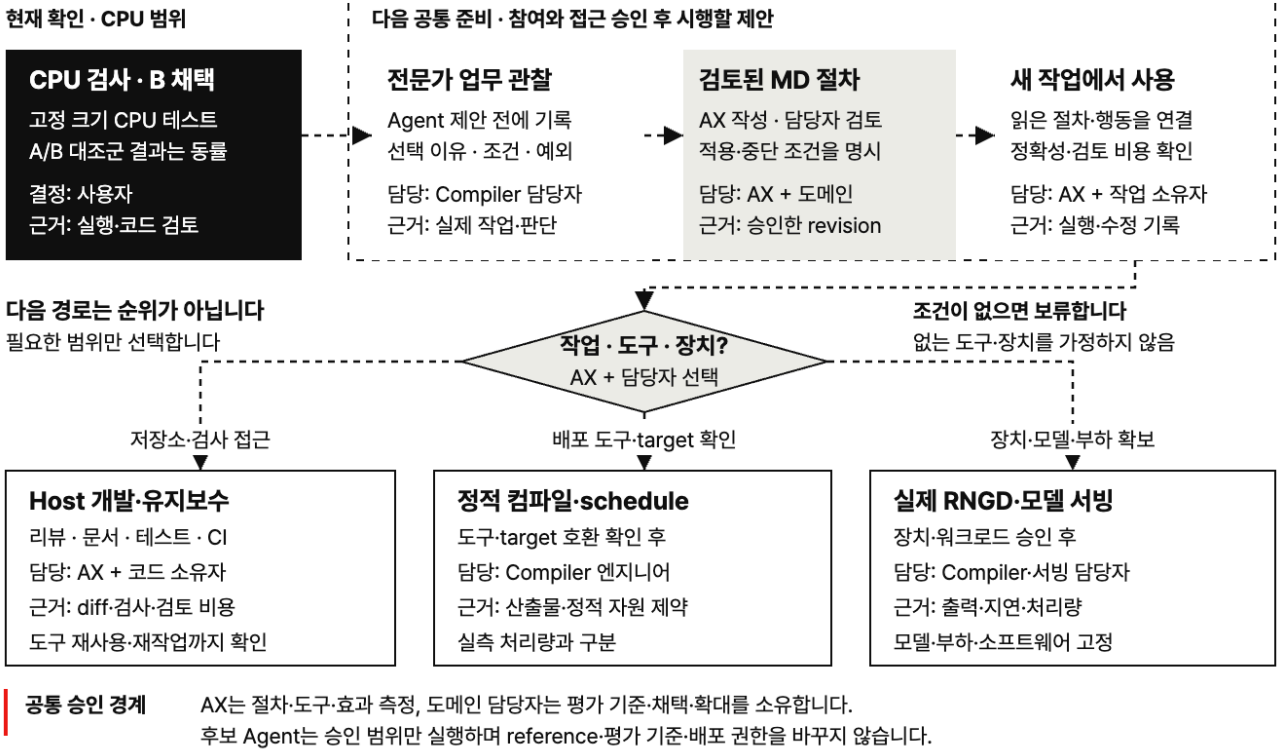


그림 6. 반복 사용이 확인된 절차는 인접 업무로 넓히고, 기술적 판단에 필요한 경로는 컴파일·장치·서빙 검사로 이어갑니다. 업무의 질문과 접근 조건에 따라 확장 경로를 선택하는 제안입니다.

업무의 질문과 접근 조건에 맞는 검증 경로에서, 다음 선택을 바꿀 작은 비교부터 만들겠습니다.

6.2 Cheap Iteration and System-Level Expansion

확장에서도 전체 작업의 결과를 기준으로 삼되, 한 번의 실험은 다음 선택에 필요한 관측을 얻을 수 있을 만큼 작게 만들겠습니다. §2에서는 두 후보의 통과 결과가 같았지만 입력 식별성을 분석해 채택할 범위를 정할 수 있었습니다. 다음 실험에서도 먼저 구별할 가설을 정하고, 이를 반복해서 확인할 입력·변경 범위·검사를 묶겠습니다. Karpathy의 autoresearch는 평가 조건을 유지하면서 제한된 코드를 수정하고 짧은 학습 실행으로 후보를 비교하는 참고가 됩니다.[13] 작은 실험에서 풀리지 않은 질문에 추가 추론이나 실행 비용을 배분하는 방식으로 시작하겠습니다.[21]

이 비교가 성립하면 추가 계산을 어디에 쓸지 정할 수 있습니다. 서로 다른 원인 가설을 독립적으로 시험할 수 있을 때는 정해 둔 총 예산과 같은 평가 조건에서 후보를 병렬로 비교하겠습니다. 국소 개선이 전체 결과에 나타나지 않을 때는 변경 범위를 다시 살펴야 합니다. 예를 들어 한 연산의 실행 시간을 줄여도 앞뒤 연산 사이의 layout 변환과 데이터 이동이 늘면 전체 실행은 빨라지지 않을 수 있습니다. 이때는 인접 연산의 배치와 buffer 수명을 함께 검토하고, 그 의존성을 바꿀 수 있는 범위로 실험을 다시 구성하겠습니다. 이렇게 비교 단위를 바꾼 뒤에도 독립적인 후보를 병렬로 시험할 수 있습니다. §4.3의 준비·검토·재작업 비용도 함께 보며, 추가 후보가 서로 다른 가설을 확인하도록 설계 범위와 실행 수를 조정하겠습니다.

설계 범위를 넓히려면 영향을 받는 산출물도 함께 갱신하고 검사할 수 있어야 합니다. Architect Labs의 Redwood 연구진은 설계 의도를 담은 ALP 명세를 중심으로 RTL·검증·펌웨어·커널을 함께 발전시키고, 필요한 관측과 실행 비용에 따라 성능 추정·RTL simulation·FPGA 실행을 사용했다고 보고합니다.[22] 이 사례는 SW·HW에 걸친 변경을 명세부터 실행까지 다시 확인할 수 있게 구성하는 방향을 보여 줍니다. Compiler AX의 확장에서도 함께 바꿀 부분과 그 영향을 확인할 검사를 연결해, 넓어진 설계 범위에서 다음 실험을 이어갈 수 있게 하겠습니다.

6.3 Replay-Based Search Policy Evaluation

작은 실험과 넓은 검증을 반복하다 보면 어느 후보부터 검사하고 어디에 더 투자할지 정하는 비용도 쌓입니다. 이때 이미 실행한 분기 기록으로 탐색 순서를 비교하는 방법을 검토할 수 있습니다. DREAM은 coding agent와 evaluator를 고정한 채 어디에서 분기하고 무엇을 병렬 실행하며 언제 멈출지를 정책 코드로 다룹니다. 기록된 tree의 결과를 재생하므로 그 범위에서는 새 discovery 실행 없이 정책을 비교합니다. 기록 밖 분기에는 새 실행이 필요합니다.[19]

이 발상은 후속 탐색 정책을 비교하는 데 필요한 기록 항목을 정해 줍니다. 부모 후보, 실제 시도, 관측, 선택·종단 이유와 비용이 있어야 나중에 다른 정책을 비교할 수 있습니다. 재생의 승자를 바로 팀 절차로 채택하지 않고 새 과제의 독립 확인을 거치겠습니다. 같은 이력의 재생 평가값 비감소는 새 업무의 개선 보장이 아니며, 정책 개발·재생·저장·검토 비용도 포함해 비교해야 합니다.

기록에 보존한 후보는 다음 실험의 출발점이 됩니다. 그 후보를 절차로 채택할 때에는 다음 과제에서 실제로 사용됐는지와 개선 효과가 확인됐는지를 구분하겠습니다. 실제 팀의 유지보수에서 유용한 작은 절차부터 남기고, 반복 사용의 근거가 쌓이는 범위에서 위임과 실행 순서를 조정하겠습니다.[20]

References and Evidence Scope

공개 자료의 구조와 개인 실행의 결과를 구분했습니다. 전문가 관찰, 팀 도입, 정적 compile/schedule과 NPU·서빙 실험은 후속 제안입니다. 공개 저장소에는 고정 A/B 소스와 공개로 전환한 오류 대조군, 선택한 과거 빌드·실행 기록, 재현 명령을 함께 두었습니다. 목차로 돌아가기

1. **GEODE**. 코드 pin dcaa764, 반복 성공 종료 수정, 개발 규약. 실행·문맥·권한의 구현과 mock 회귀 근거입니다. 규약 존재를 준수율·생산성 측정으로 계산하지 않습니다.
2. **Eco²**. Backend pin 4c3eebb, ACK 정책 수정. Consumer 전후 대조는 격리 재현이며 서비스 통합 재측정과 다릅니다.
3. **SIL·Crucible**. Seed Generation, Scaffold search, 기각 후 복원 수정, Petri 공식 소개. 일부 seed는 모의 응답을 사용하며 search 재평가는 옵션입니다. 설계·수정 근거와 자동 실행 이력을 구분합니다.
4. **Harbor 비교의 날짜와 모집단**. Harbor 0.22.0, 로컬 terminalbench21-sol-max-fullsuite-paired-20260827t190300z, analysis SHA 앞 12자리 c32619e98bb3, 2026-09-12 확인. amd64 검증기 두 과제를 대칭 제외한 뒤에도 native 측 6셀에 인프라 무효가 남았습니다. 공통 유효 429셀에서 GEODE 339·Codex 331 통과는 보조 관측이며 전체 평가값이나 인과적 우위가 아닙니다. 공개 평가 저장소는 안내 경로이며 이 로컬 분석의 공개 완료 증거와 다릅니다.
5. **Compiler AX Lab**. 고정 A/B 소스·실행 증거, 이전 예제·검사, B 검토·채택 기록, CI 구현. SDK 9b9cf0fdc78df00cdc430eae725a5ad9084a735e, nightly 2026-05-01. 검토는 OpenAI Codex / gpt-6-astra / ultra, 생성 요청은 high입니다. B는 개인 CPU 실험의 후속 코드로 채택했습니다. 같은 Device 호출도 물리 HBM 재사용을 보장하지 않습니다.
로컬 최종 산출물 825개의 무결성 확인 뒤, 공개에 필요한 102개 파일을 선별했습니다. 원본 98개와 공개용 설명·요약·검사기·manifest로, 소스·바이너리 hash·명령·출력·종료 상태를 연결합니다. 공개 후에도 과거 실행 기록과 새 재현을 구분하며, 공개된 오류는 후속 비공개 평가에 재사용하지 않습니다. 이전 공개 테스트·parser 변경의 별도 통합 실행은 일반 테스트 720개와 doctest 55개를 통과했습니다. 이 수치는 B의 통합 결과가 아닙니다. Dctest 44개는 compile_fail이며 ignored test 17개는 제외했습니다. 환경은 Ubuntu 24.04 amd64/Rosetta, 2 CPU·6 GiB, Cargo jobs=1입니다. Source-clean parser·합성 상태 복원·native smoke는 별도 모집단입니다.
6. **TCP·RNGD 데이터 흐름**. Hanjoon Kim 외, TCP: A Tensor Contraction Processor for AI Workloads, ISCA 2024, §II-IV-V, Fig.2-3-8-9. Younggeun Choi 외, FuriosaAI RNGD: A Tensor Contraction Processor for Sustainable AI Computing, IEEE Micro 45(3), 2025, 저자본 1-6쪽. 당시 탐색 알고리즘과 현재 내부 pass는 구분합니다.
7. **Programming model과 공개 검사**. RENEGADE 2026 01:30:44, SDK 2026.3, 0.8.1 kernel validation, 공개 driver. 구조 compile-plain Cargo CPU 값·schedule·NPU 검사는 별도 근거입니다. 공개 Rust 경로를 내부 전체 구현과 동일시하지 않습니다.
8. **실행·산출물 책임**. Software Stack, FXB, Runtime, 장치 접근 조건, Furiosa-LLM serve. 호환성·실제 출력·서비스 SLO를 분리합니다.

9. **변환의 보존 조건.** Cornell CS4120 IR Lowering과 Arm TOSA 1.0.2 §1.10. 값·효과·평가 순서와 검사 비완전성의 원리를 사용하며 특정 TCP 변환의 증명으로 대체하지 않습니다.
10. **모델 적용 사례.** Solar on Daum, 2026-07-31; EXAONE 3.5 공동 배포, 2025-07-22; SDK 2026.2, ArtifactBuilder, Hybrid KV. 협업 사례와 EXAONE 4 후속 변경을 구분합니다. Bucket·KV 변경의 기여는 후속 분리 실험으로 확인하겠습니다.
11. **운영 계층.** 삼성SDS NPUaaS, 2026-07-20; Lablup-Furiosa RNGD meets Backend.AI, July 2026 v1.0. 자원 배정·세션·서빙 책임의 참고이며 내부 장애 빈도·인력 공수의 근거로 쓰지 않습니다.
12. **Compiler AX 역할.** 공식 채용 공고, 2026-09-16 확인. 실제 업무의 실험·도구 정착·playbook·온보딩과 PR cycle/review/triage/flakiness의 영향 측정을 제시합니다. 구체 내부 병목의 우선순위는 관측으로 확인할 항목입니다.
13. **생성·실험 설계.** Google Research, Towards an AI co-scientist §3.3-4.1; Google DeepMind, AlphaEvolve §2-4; Andrej Karpathy, autoresearch README·program.md, pin 228791f. 역할 분리·실행 평가·고정 비교의 근거입니다. autoresearch는 prepare.py의 준비·평가를 고정하고 train.py를 수정해 val_bpb를 비교합니다. 5분은 학습 예산이며 준비·컴파일을 포함한 전체 작업시간과 구분합니다.
14. **검사와 관측의 신뢰성.** LLVM Benchmarking, OSS-Fuzz target generation, SQLite testing. LLVM의 반복 측정·환경 잡음 통제, OSS-Fuzz의 생성 검사 대상에 대한 컴파일·API 사용·coverage 확인, SQLite의 결함 재현 검사를 참고했습니다. §5.1에서는 실행 성립 여부와 수치 판정을 나눠 적용했습니다.
15. **선택적 데이터 조회.** AWS Bedrock AgentCore RLM, 2026-05-21; Redis HGETALL·HSCAN. 큰 묶음의 조회 비용을 참고합니다. HSCAN COUNT는 엄격한 바이트 상한이 아닙니다. 이 조회 비용을 고려해 §5.3의 manifest·완전성 확인·보존 방식을 설계했습니다.
16. **Dioxus의 검증과 완료 범위.** Jonathan Kelley, Building ambitious software, 2026-09-11, 보존 자동 전사 06:18-09:53:10:47-11:14:14:21-18:08. Jonathan Kelley의 운영 경험을 참고했습니다. Head/base 확인과 별도 release 승인은 §5.4의 변경 검토 절차에 추가했습니다.
17. **정적 스케줄 진단.** furiosa-opt pin 9b9cf0f의 Diagnosis-Tuning. 마지막 완료를 결정하는 구간의 의존성·자원·빈 구간을 구분하고 한 선택씩 비교합니다. 이번 CPU 파일럿의 완료 범위에 포함되지 않습니다.
18. **현재 칩과 후속 계획.** RNGD 사양, Broadcom 협력 발표, 2026-05-27; 회사 보도자료 재배포의 2028년 상반기 sampling 계획. 공개된 compute/I/O die 분리·HBM4/4E·scale-up 통신은 새로 확인할 데이터 이동·실행 조건의 근거입니다. 미래 칩의 구현·ABI·RNGD 호환성·탐색 공간 증가량은 공개 확인된 결과로 쓰지 않습니다.
19. **DREAM.** Tong Zheng 외, Dream-RSI: Recursive Self-Improvement through Evolving Worlds, 2026-09-14, §3-5; 공식 사이트, 저장소, 9월 16일 확인. 현재 정책을 후보에 포함한 같은 이력의 평균 재생 목적함수에서 비감소합니다. 새 업무·실제 지연·총비용 보장은 별도입니다. 사이트 시연은 설명용 수치이며 전체 실행·재현 코드는 공개 준비 상태입니다. 기록 tree를 재생하는 방법과 조건부 경로의 시각 원리를 참고했습니다.
20. **후속 개선의 분류.** Zhang 외, Darwin Gödel Machine v3, §3-4·Appendix H 및 archive 선택. 후보 보존과 부모 대비 개선 채택은 다릅니다. The Last AI Built by Humans v2, §3.6-3.7·Fig.8은 지속되는 변경·개선 권한·외부 제약을 구분하는 참고입니다. 설명용 미래 곡선을 전망치로 사용하지 않았습니다.
21. **개인 LLM 사용의 비용과 확인.** Andrej Karpathy, How I use LLMs, 2025-02-27. 자동 자막 30:10-30:25의 빠른 모델→추론 모델 선택, 1:06:00-1:06:37의 생성 코드 가정 확인을 참고했습니다. 빠른 답에서 추가 추론으로 전환하는 판단을 참고했습니다. 실험의 실행비 배분은 §4.3의 비교 기준에 맞춰 설계하겠습니다.
22. **SW-HW 설계·검증 사례.** Architect Labs Team, Redwood v2, 2026-08-28, §I·IV·V·VI, 1-3-7쪽, Table I; 연구진의 프로젝트 설명, 2026-08-27. 연구진은 기존 ALP·모델·EDA 기반 위에서 두 architect가 2주 이내 설계·검증·FPGA 배포를 마치고, 3주 차에 Qwen3-0.6B를 연결했다고 보고합니다. AMD Versal VPK180의 2×2 tile-250 MHz 구성에서 생성 128 tokens 평균은 12.1 token/s이며 host 전송을 포함합니다. ASIC은 physical design-tapeout-post-silicon 검증이 남은 후속 작업입니다. 연구진은 FPGA의 Qwen3 endpoint를 후속 accelerator의 timing·kernel 개선에 사용한 시도를 초기 RSI로 설명합니다.